

CS590 FINAL REPORT - VYNN AI

Zanwen Fu

Department of Computer Science
Duke University

ABSTRACT

Financial decision-making increasingly depends on timely interpretation of unstructured textual information such as news, disclosures, and market commentary. While large language models (LLMs) offer powerful semantic capabilities, naïve prompting is insufficient for financial analysis, which demands grounded data access, transparent computation, and reproducible numerical reasoning. We present **VYNN AI** (Value Your Next News), a production-grade multi-agent system that integrates text mining, deterministic valuation modeling, and real-time market monitoring within a structured execution framework.

VYNN AI decomposes financial analysis into specialized agents coordinated by a LangGraph-based Supervisor that combines rule-constrained symbolic routing with LLM-mediated semantic interpretation. Unstructured news is transformed into evidence-grounded events, mapped to valuation parameters, and propagated through a fully symbolic discounted cash-flow model, ensuring auditability and reproducibility. The system is deployed using an isolated supervisor-worker architecture with cache-aware fan-out, enabling scalable execution under overlapping user queries.

We evaluate VYNN AI on real production workloads and market events, demonstrating stable routing behavior, exact reproducibility of symbolic outputs, and substantial cache-amortized latency reductions. Our results highlight how agentic architectures can bridge unstructured text and structured financial reasoning, and more broadly illustrate a transferable pattern for building reliable, interpretable text-mining systems in high-stakes domains.

1 INTRODUCTION

Financial markets operate under continuous and accelerating information flow, where corporate disclosures, macroeconomic indicators, geopolitical developments, and real-time news jointly shape asset prices. Prior work shows that market narratives disseminated through media influence investor beliefs and trading behavior, while the growing scale of textual disclosures increases the burden of extracting actionable signals Tetlock (2007); Loughran & McDonald (2011). Professional analysts face substantial synthesis costs when combining earnings materials, market data, and narrative context Brown et al. (2015), while retail investors navigate fragmented platforms that separate news, fundamentals, and analytical tools. The result is persistent information asymmetry and delayed reactions to market-moving events, particularly during periods of heightened volatility when early interpretation confers disproportionate advantage Hong et al. (2000).

Existing financial technologies only partially address this challenge. Institutional platforms such as Bloomberg Terminal and LSEG Workspace offer comprehensive data and analytics but remain expensive and heavily reliant on manual interpretation. Consumer-facing platforms prioritize accessibility but provide limited analytical depth, often leaving users to infer causal links between news and valuation. Recent advances in large language models (LLMs) promise systems that can read unstructured text, retrieve structured data, and synthesize insights. However, naïve LLM prompting is insufficient for financial reasoning: markets require grounded data access, transparent computation, long-horizon reasoning, and consistent tool usage—properties not reliably guaranteed by standalone LLM inference Mialon et al. (2023); Bommasani et al. (2021); Huang et al. (2023).

Agentic AI systems, in which LLMs coordinate external tools and multi-step workflows, offer a compelling alternative. Recent research demonstrates that LLM-based agents excel in tasks requiring

planning and iterative refinement Yao et al. (2023); Shinn et al. (2023). Yet their application to financial modeling and real-time market analysis remains underexplored. Finance is a particularly challenging testbed: effective analysis requires integrating unstructured textual signals (news), structured numerical data (fundamentals), symbolic computation (valuation models), and temporal dynamics (market updates). This confluence motivates architectures that combine semantic flexibility with explicit correctness guarantees.

To address this challenge, we introduce **VYNN AI** (Value Your Next News), a multi-agent system designed to monitor financial information, interpret its relevance, update valuation models, and surface timely insights. At its core is a LangGraph-based Supervisor that decomposes user queries into structured subtasks and delegates them to specialized agents for news screening, financial data retrieval, discounted cash-flow (DCF) valuation, parameter adjustment from textual events, and evidence-grounded narrative synthesis. Unlike conventional LLM chatbots, VYNN AI operates as an autonomous analysis pipeline: it retrieves verifiable data, executes deterministic numerical models, produces structured artifacts, and guarantees reproducibility through isolated execution.

The system is implemented as a two-tier production architecture comprising a lightweight FastAPI controller and a fleet of ephemeral worker containers. The controller manages authentication, caching, and job scheduling, while workers execute agentic workflows in isolation to ensure fault containment and reproducibility. MongoDB provides durable storage and Redis enables low-latency caching, allowing the system to eliminate redundant analyses when users query overlapping companies or news events.

Although the primary research focus is the agentic analysis engine, we also implement a complementary broker-style dashboard that contextualizes agent outputs within practical investment workflows. The dashboard surfaces news feeds, daily summaries, stock cards, and portfolio visualizations powered directly by artifacts generated by the agentic backend, illustrating how agent-based systems can integrate into real user-facing financial tools.

In summary, this work makes three contributions:

- We propose a multi-agent architecture for financial analysis that fuses text mining, deterministic valuation modeling, and real-time market monitoring.
- We design and implement a scalable supervisor–worker execution framework that enables structured, reproducible, and tool-augmented agentic reasoning.
- We evaluate the system through production experiments and real-market case studies, demonstrating cache-amortized latency reductions, symbolic reproducibility, and qualitative robustness.

By unifying agentic AI with text-mining-driven financial reasoning, VYNN AI demonstrates how emerging AI architectures can reduce information latency and broaden access to institutional-grade financial analysis.

2 BACKGROUND AND RELATED WORK

VYNN AI lies at the intersection of financial text mining, tool-augmented large language models (LLMs), agentic orchestration, and symbolic valuation. We review these areas with a narrow goal: situating VYNN AI relative to prior systems and clarifying the architectural gap it addresses. Existing approaches often excel at (i) extracting signals from financial text or (ii) performing multi-step tool use with LLMs, but rarely provide a reproducible, valuation-aware bridge from unstructured real-time information to structured financial decision-making under explicit correctness constraints.

2.1 FINANCIAL TEXT MINING AND MARKET SIGNALS

A substantial literature studies how text—including news, filings, and financial communications—relates to market behavior. Classic results show that media tone and domain-specific language patterns correlate with returns and volatility Tetlock (2007); Loughran & McDonald (2011). More recent work applies transformer-based models to financial text to capture richer semantic representations for sentiment and event-related tasks Araci (2019); Yang et al. (2020).

Most financial NLP systems remain prediction-oriented, producing sentiment scores or forecasts that are difficult to audit and do not directly correspond to the interpretable mechanisms used in fundamental analysis. VYNN AI instead targets decision-ready representations by grounding extracted events in evidence spans and mapping them to explicit valuation levers (e.g., growth, margins, discount-rate components) that can be propagated through a transparent model.

2.2 RETRIEVAL AND TOOL AUGMENTATION FOR GROUNDED LLMs

Retrieval-augmented generation (RAG) improves factuality by grounding LLM outputs in external sources Lewis et al. (2020). Tool-augmented LLMs further extend capabilities by learning to invoke APIs and computational tools Schick et al. (2023). These paradigms enable knowledge-intensive and tool-mediated tasks, but many deployments still rely on single-prompt pipelines or loosely structured tool-call sequences, limiting intermediate verification and reproducibility—especially for numerical reasoning.

VYNN AI adopts retrieval and tool use within an explicit orchestration graph: intermediate artifacts (events, financial tables, parameter updates, valuation outputs) are represented as structured state rather than implicit prompt context, and all numerical computation is executed deterministically outside the LLM.

2.3 AGENTIC ORCHESTRATION AND SYMBOLIC VALUATION

Reasoning-and-acting agents such as ReAct demonstrate that LLMs can iteratively plan and invoke tools Yao et al. (2023), and subsequent work explores agents that improve via self-reflection and iterative correction Shinn et al. (2023). However, prompt-centric control often makes termination, traceability, and correctness difficult to guarantee. In parallel, discounted cash-flow (DCF) valuation remains a cornerstone of fundamental analysis due to its interpretability and theoretical grounding Damodaran (2012); Penman (2013), but is traditionally manual and slow to update under new information.

VYNN AI combines these threads by enforcing a strict semantic–symbolic separation: LLMs are restricted to semantic tasks (intent recognition, relevance assessment, event extraction), while valuation and parameter propagation are implemented symbolically for reproducibility and auditability. This design yields a system that can ingest real-time text, produce evidence-grounded structured signals, and update valuation outputs deterministically, while scaling through cache-aware fan-out.

2.4 POSITIONING

Relative to RAG summarization and tool-calling report generators, VYNN AI contributes three architectural commitments: **(i)** explicit state semantics for verifiable intermediates, **(ii)** symbolic correctness for numerical computation, and **(iii)** cache-aware orchestration to amortize repeated analyses across overlapping user requests. Together, these properties operationalize the “Value Your Next News” objective: translating real-time unstructured information into structured, valuation-aware financial insights.

3 SYSTEM OVERVIEW

VYNN AI is implemented as a modular production system that integrates agentic reasoning, deterministic financial modeling, and real-time market data within a scalable execution framework. The system architecture, shown in Figure 1, consists of a lightweight controller service, a pool of ephemeral worker containers, and a persistent data layer shared by both an agentic chatbot and a broker-style dashboard. This design follows established distributed-systems principles: separating control from computation, isolating execution to ensure fault containment, and leveraging caching to amortize expensive operations Dean & Barroso (2013); Sigelman et al. (2010).

3.1 CONTROLLER AND WORKER EXECUTION

The controller layer is implemented as a FastAPI service that mediates all external interactions, including chatbot queries, valuation requests, and dashboard operations. It performs request parsing,

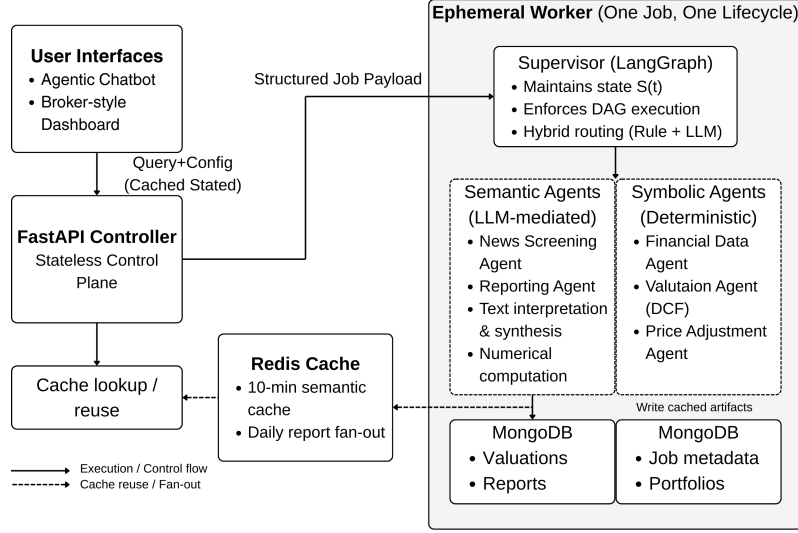


Figure 1: High-level system architecture of VYNN AI. User requests are handled by a stateless FastAPI controller, which performs cache lookup and dispatches isolated ephemeral workers. Each worker executes a supervisor-orchestrated multi-agent workflow and persists artifacts to MongoDB, while Redis enables cache-aware fan-out and reuse across repeated or overlapping queries.

authentication, job creation, cache lookup, and orchestration of worker execution. Upon receiving a request, the controller resolves reusable artifacts from Redis, persists job metadata to MongoDB, and launches an ephemeral worker with a structured payload describing the query, configuration, and relevant cached state. Crucially, the controller performs no heavy computation, remaining highly available under load Newman (2015).

Each worker is an isolated Docker container launched to execute exactly one agentic workflow and is terminated upon completion. Inside the worker, a LangGraph-based Supervisor coordinates multi-step execution across subagents, including news screening, financial data retrieval, valuation modeling, parameter adjustment, and report generation. Because workers execute deterministically against fixed inputs, failures are confined to individual jobs and reproducibility is guaranteed. Workers stream structured logs and state updates to the controller during execution, persist final artifacts to MongoDB, and exit cleanly, avoiding cross-job contamination. This ephemeral execution model aligns with best practices in containerized batch and serverless systems Burns et al. (2016).

3.2 DATA, CACHING, AND INGESTION

The data layer underpins both correctness and efficiency. MongoDB serves as the durable store for historical analyses, valuation outputs, report artifacts, job metadata, and portfolio information. Redis provides low-latency caching for repeated computations, job status, recent news clusters, and real-time market snapshots. Together, these stores enable cache-aware fan-out semantics: repeated or overlapping requests reuse validated artifacts rather than re-triggering full agentic execution, bounding redundant computation and tail latency Dean & Barroso (2013).

Complementing these stores is the `vynn-core` ingestion service, a continuously running background process that collects news from external sources, normalizes text, deduplicates URLs, and maintains hot feeds consumed by workers at runtime. By decoupling ingestion from analysis, the system ensures that agent execution operates on temporally fresh yet internally consistent market information Kreps et al. (2011).

3.3 USER INTERFACES AND DESIGN SUMMARY

VYNN AI exposes two user-facing surfaces. The agentic chatbot invokes the full multi-agent pipeline through the controller’s job API, surfacing intermediate logs, extracted insights, valuation tables, and synthesized narratives. In parallel, a broker-style dashboard consumes artifacts generated by

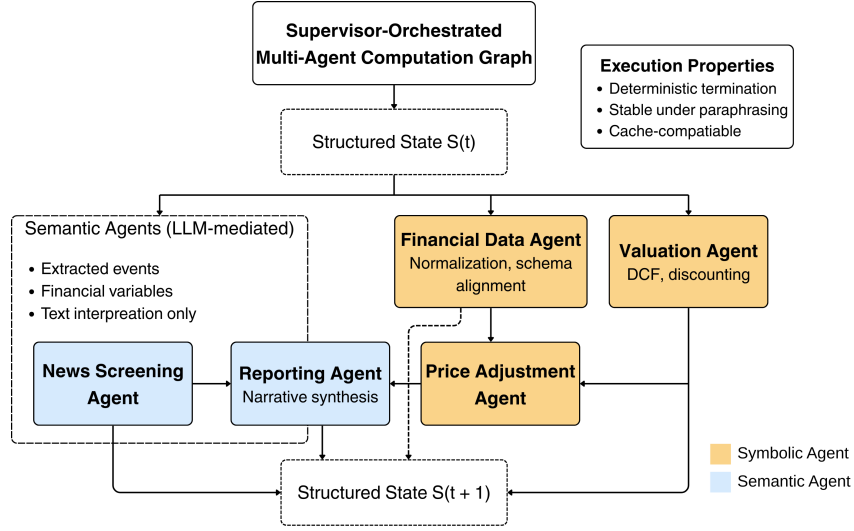


Figure 2: Supervisor-orchestrated multi-agent computation pipeline. A LangGraph-based Supervisor maintains a structured state and coordinates semantic agents (LLM-mediated) and symbolic agents (deterministic) within a directed acyclic execution graph. Semantic agents perform text interpretation and narrative synthesis without numerical computation, while symbolic agents execute financial data normalization, valuation, and bounded parameter adjustment. Explicit state passing ensures deterministic termination, reproducibility, and stability under paraphrasing.

prior agentic runs and real-time cached data to support portfolio monitoring, daily report viewing, and exploratory analysis. Although the dashboard itself is not agentic, it demonstrates how agent-generated artifacts integrate naturally into analyst workflows without duplicating computation.

Overall, VYNN AI’s architecture reflects three design priorities: (i) deterministic and auditable agent execution via explicit state and ephemeral isolation, (ii) horizontal scalability through containerized worker parallelism, and (iii) low-latency responsiveness enabled by cache-aware orchestration and continuous ingestion. Together, these choices provide a practical and extensible foundation for automated financial intelligence.

4 METHODOLOGY: MULTI-AGENT PIPELINE

VYNN AI performs financial analysis through a structured multi-agent computation pipeline that integrates semantic text interpretation, structured data retrieval, symbolic valuation modeling, and evidence-grounded narrative synthesis. Execution is orchestrated by a LangGraph-based Supervisor that coordinates specialized subagents through a hybrid symbolic–neural routing policy. This design enforces determinism, interpretability, and execution stability while enabling flexible semantic reasoning. Figure 2 illustrates the end-to-end workflow.

4.1 AGENTIC COMPUTATION GRAPH

The pipeline is modeled as a directed acyclic computation graph

$$G = (V, E),$$

where each node $v \in V$ corresponds to a subagent and each directed edge $(v_i \rightarrow v_j) \in E$ denotes an admissible execution transition. Acyclicity guarantees termination after at most $|V|$ agent invocations.

At execution step t , the Supervisor maintains a structured state

$$s_t = \{s_t^{\text{news}}, s_t^{\text{fin}}, s_t^{\text{proj}}, s_t^{\text{val}}, s_t^{\text{adj}}, s_t^{\text{nar}}, s_t^{\text{cache}}, s_t^{\text{flags}}\},$$

encoding extracted events, normalized financials, valuation outputs, parameter adjustments, narrative artifacts, cache metadata, and control flags. All symbolic components are deterministic, ensuring

reproducibility under identical inputs and stable routing under semantically equivalent prompts, as evaluated in Section 5.

The Supervisor selects actions from

$$\mathcal{A} = \{a^{\text{news}}, a^{\text{fin}}, a^{\text{val}}, a^{\text{adj}}, a^{\text{rep}}, a^{\text{halt}}\},$$

using a hybrid routing policy

$$a_t = \pi(s_t) = \begin{cases} \pi_{\text{rule}}(s_t), & \text{if structural dependencies apply,} \\ \pi_{\text{LLM}}(s_t), & \text{if semantic interpretation is required.} \end{cases}$$

Symbolic routing enforces execution correctness (e.g., valuation cannot precede data retrieval), while LLM-based routing is restricted to semantic decisions such as intent classification and event relevance. LLMs are never used for numerical computation.

4.2 SEMANTIC AGENTS

News Screening Agent. The News Screening Agent converts raw financial news into structured events. Articles are normalized, deduplicated, and analyzed to extract salient events, each classified by type, directional impact, and confidence and explicitly grounded to supporting text spans. Each event is represented as

$$(s_t^{\text{news}})_i = (e_i, d_i, c_i, \tau_i),$$

where e_i denotes event type, d_i directional impact, c_i confidence, and τ_i the evidential span. This explicit text-to-factor mapping enables interpretable integration of linguistic signals into valuation logic.

Reporting Agent. The Reporting Agent synthesizes analytical narratives describing extracted catalysts, valuation rationale, updated price targets, and risks. While generation leverages LLM reasoning, all claims must reference structured state entries; unsupported statements are rejected by a symbolic validator. Reports are rendered deterministically into structured formats. In deployment, the agent can operate in a daily-report mode, producing one company-level report reused across users (Section 5).

4.3 SYMBOLIC AGENTS

Financial Data Agent. The Financial Data Agent retrieves historical fundamentals and market metadata, applying deterministic schema alignment, temporal consistency checks, and conservative imputation for missing values. All interventions are logged in the state flags. This agent is intentionally lightweight and contributes negligible runtime relative to semantic agents (Section 5).

Valuation Agent. Valuation is performed using a fully symbolic discounted cash-flow (DCF) model. Free cash flow is computed as

$$\text{FCF}_t = \text{NOPAT}_t + \text{D\&A}_t - \text{CapEx}_t - \Delta\text{NWC}_t.$$

The discount rate is

$$r = r_f + \beta \cdot \text{ERP},$$

and terminal value is

$$TV = \frac{\text{FCF}_{T+1}}{r - g}.$$

Enterprise value is given by

$$EV = \sum_{t=1}^T \frac{\text{FCF}_t}{(1+r)^t} + \frac{TV}{(1+r)^T}.$$

All intermediate and final outputs are stored in s_t^{val} , enabling exact reproduction and auditability.

Price Adjustment Agent. Text-derived events perturb valuation parameters θ_k (e.g., growth, margins, discount rate) via

$$\theta'_k = \theta_k + \Delta_k(e_i, d_i, c_i),$$

where Δ_k satisfies monotonicity with respect to event direction, bounded magnitude $|\Delta_k| \leq \epsilon_k$, and Lipschitz continuity in confidence. These constraints ensure stable propagation of textual uncertainty.

4.4 DESIGN RATIONALE AND PROPERTIES

Design motivations and execution guarantees underlying the pipeline—including the rejection of monolithic prompting, avoidance of LLM-only valuation, and system-level determinism—are discussed in Appendix C.

5 EVALUATION AND RESULTS

We evaluate VYNN AI as an agentic systems pipeline rather than a predictive model, emphasizing end-to-end latency, cache-amortized efficiency, reproducibility, and qualitative correctness on real market events. This systems-centric framing is motivated by two observations: production latency variability often dominates user experience at scale Dean & Barroso (2013), and reliability in distributed pipelines requires explicit instrumentation and traceability Sigelman et al. (2010). All results are obtained from actual production runs of the deployed backend; we report no synthetic workloads or simulated data.

5.1 EVALUATION SCOPE

The evaluation addresses four questions: (RQ1) end-to-end latency characteristics, (RQ2) cache-amortized efficiency via fan-out, (RQ3) reproducibility and stability under paraphrasing, and (RQ4) qualitative correctness on real market events. Reproducibility is defined as exact equality of symbolic valuation outputs under identical inputs, while stability under paraphrasing denotes consistency of Supervisor routing and extracted event representations across semantically equivalent prompts.

5.2 EXPERIMENTAL SETUP

All experiments run on the production deployment. Each request executes inside an isolated worker container coordinated by the Supervisor, with timing captured at agent boundaries following distributed tracing principles Sigelman et al. (2010). We conduct four experiments: (1) end-to-end latency and per-agent breakdown (five workflows), (2) cache-aware efficiency computed from measured agent times, (3) reproducibility and paraphrase stability (nine runs across three tickers), and (4) qualitative case studies on real market events (three cases; 35 articles). A detailed summary of all evaluation experiments is provided in Appendix B.3.

5.3 LATENCY AND AGENT BREAKDOWN (RQ1)

Observed end-to-end latency ranges from **20.0 s** for minimal workflows to **383.0 s** for full four-agent executions. Across five runs, the mean latency is **152.5 s** and the median is **98.0 s**. Variance is driven primarily by workflow complexity and news volume rather than numerical computation, consistent with prior observations that semantic processing dominates LLM-mediated pipelines Kwon et al. (2023).

For the full META workflow (383.0 s total), semantic agents account for the majority of runtime: News Screening consumes **189.4 s** (49.4%) and Reporting **167.6 s** (43.8%), while the Financial Data and Valuation agents together account for under **3%**. Supervisor overhead contributes approximately **4.2%**. Figure 3 illustrates how end-to-end latency varies across workflow configurations, highlighting the dominant cost of semantic processing and the impact of modular execution. A detailed per-agent runtime decomposition supporting this observation is provided in Appendix B.

5.4 CACHE-AMORTIZED EFFICIENCY (RQ2)

VYNN AI employs short-term (10-minute) caching for News Screening and Reporting outputs, as well as daily fan-out for company-level reports. Rather than reporting synthetic cache hit rates, we compute an effective cached runtime by removing measured News Screening time, reflecting subsequent executions that reuse cached artifacts.

For the full META workflow, removing the **189.4 s** News Screening stage reduces effective latency from **383.0 s** to **193.6 s**, corresponding to an approximate **49%** reduction. These reductions follow

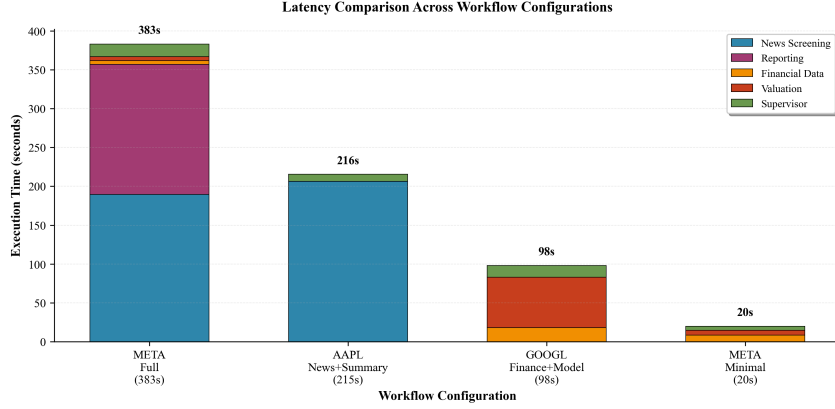


Figure 3: End-to-end latency comparison across workflow configurations. Full pipelines incur higher latency due to semantic processing, while partial and cached workflows significantly reduce execution time. The results highlight how modular agent execution and cache-aware fan-out amortize expensive semantic operations without affecting symbolic correctness.

directly from measured agent timings and illustrate how fan-out semantics bound redundant computation and mitigate tail amplification under repeated queries Dean & Barroso (2013). A controlled cache robustness study is deferred to future work. Additional breakdowns illustrating the separation between semantic, symbolic, and coordination overheads are reported in Appendix B.

5.5 REPRODUCIBILITY AND STABILITY (RQ3)

Across **9** runs spanning NVDA, AAPL, and MSFT, VYNN AI achieves **100% reproducibility** of symbolic outputs, including valuation parameters and intrinsic values. Within-ticker execution-time variability is low (coefficient of variation between **0.016** and **0.035**), while cross-ticker differences reflect semantic workload rather than nondeterminism.

Paraphrase robustness tests on NVDA show consistent Supervisor routing, agent invocation order, and extracted events across semantically equivalent prompts. The measured stability score exceeds **0.98**, indicating robustness to surface-level linguistic variation.

5.6 QUALITATIVE CASE STUDIES (RQ4)

We evaluate qualitative correctness on three real market cases (META, NVDA, AAPL), processing **35** news articles. Reports are assessed for event extraction correctness, text-to-valuation coherence, and evidence grounding. META and NVDA execute full pipelines and produce coherent analyses linking extracted catalysts to valuation adjustments, while AAPL leverages cached daily-report execution. Minor serialization gaps appear in intermediate artifacts despite correct final outputs; we treat these as engineering issues rather than conceptual failures and discuss them in Appendix A.

5.7 SUMMARY

Across all experiments, a consistent systems-level pattern emerges. Semantic agents dominate end-to-end latency due to the inherent cost of large-scale text interpretation, while symbolic components remain fast and deterministic. Cache-aware fan-out policies substantially amortize semantic overhead for repeated and overlapping queries, yielding large effective latency reductions without compromising correctness. At the same time, the explicit separation between semantic interpretation and symbolic valuation guarantees exact reproducibility of numerical outputs and stable routing behavior under paraphrased inputs. Qualitative case studies further demonstrate that these properties translate into coherent, evidence-grounded analyses on real market events. Taken together, these results validate VYNN AI as a robust, interpretable agentic framework for transforming real-time unstructured text into structured, valuation-aware financial insights.

CONTRIBUTIONS AND ARTIFACTS

Contributions. This project was completed individually. The author was responsible for the end-to-end design, implementation, experimentation, and evaluation of the VYNN AI system, including the multi-agent architecture, supervisor logic, symbolic valuation pipeline, caching and execution infrastructure, frontend integration, and all experimental analyses reported in this paper. The author also wrote the manuscript in its entirety.

Code and Artifacts. The full implementation of VYNN AI, including the agentic backend, supervisor-worker execution framework, data ingestion components, and supporting infrastructure, is publicly available at:

<https://github.com/Agentic-Analyst>

The repository includes documentation and instructions for reproducing the experiments and system behavior described in this paper.

Demonstrations. Live demo, example outputs, and a gallery of system screenshots illustrating the agentic workflows and user-facing dashboard are available at:

<https://vynnai.com/demo>

REFERENCES

- Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in ai safety. arXiv preprint arXiv:1606.06565, 2016.
- Dogu Araci. Finbert: Financial sentiment analysis with pre-trained language models. arXiv preprint arXiv:1908.10063, 2019.
- Rishi Bommasani, Drew A. Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S. Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, et al. On the opportunities and risks of foundation models. arXiv preprint arXiv:2108.07258, 2021.
- Lawrence D. Brown, Andrew C. Call, Michael B. Clement, and Nathan Y. Sharp. Inside the “black box” of sell-side financial analysts. Journal of Accounting Research, 53(1):1–47, 2015.
- Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, omega, and kubernetes. Communications of the ACM, 59(5):50–57, 2016.
- Aswath Damodaran. Investment Valuation: Tools and Techniques for Determining the Value of Any Asset. Wiley, 3 edition, 2012.
- Jeffrey Dean and Luiz André Barroso. The tail at scale. Communications of the ACM, 56(2):74–80, 2013. doi: 10.1145/2408776.2408794.
- Harrison Hong, Terence Lim, and Jeremy C. Stein. Bad news travels slowly: Size, analyst coverage, and the profitability of momentum strategies. The Journal of Finance, 55(1):265–295, 2000.
- Xiaowei Huang, Wenjie Ruan, Wei Huang, Gaojie Jin, Yi Dong, Changshun Wu, Saddek Bensalem, Ronghui Mu, Yi Qi, Xingyu Zhao, Kaiwen Cai, Yanghao Zhang, Sihao Wu, Peipei Xu, Dengyu Wu, André Freitas, and Mustafa A. Mustafa. A survey of safety and trustworthiness of large language models through the lens of verification and validation. arXiv preprint arXiv:2305.11391, 2023.
- Jay Kreps, Neha Narkhede, and Jun Rao. Kafka: A distributed messaging system for log processing. In Proceedings of the NetDB Workshop, pp. 1–7, 2011.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In Proceedings of the 29th Symposium on Operating Systems Principles (SOSP), pp. 611–626, 2023.

- Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. In Advances in Neural Information Processing Systems, volume 33, pp. 9459–9474, 2020.
- Tim Loughran and Bill McDonald. When is a liability not a liability? textual analysis, dictionaries, and 10-ks. The Journal of Finance, 66(1):35–65, 2011.
- Grégoire Mialon, Roberto Dessì, Maria Lomeli, Christoforos Nalmpantis, Ram Pasunuru, Roberta Raileanu, Baptiste Rozière, Timo Schick, Jane Dwivedi-Yu, Asli Celikyilmaz, et al. Augmented language models: A survey. arXiv preprint arXiv:2302.07842, 2023.
- Sam Newman. Building Microservices. O’Reilly Media, 2015.
- Stephen H. Penman. Financial Statement Analysis and Security Valuation. McGraw-Hill, 5 edition, 2013.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, et al. Toolformer: Language models can teach themselves to use tools. In Advances in Neural Information Processing Systems, volume 36, 2023.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In Advances in Neural Information Processing Systems (NeurIPS), volume 36, 2023.
- Benjamin H. Sigelman, Luiz André Barroso, Mike Burrows, Pat Stephenson, Manoj Plakal, Donald Beaver, Saul Jaspan, and Chandan Shanbhag. Dapper, a large-scale distributed systems tracing infrastructure. Technical Report 2010-1, Google, Inc., 2010.
- Paul C. Tetlock. Giving content to investor sentiment: The role of media in the stock market. The Journal of Finance, 62(3):1139–1168, 2007.
- Yi Yang, Mark Christopher Siy Uy, and Allen Huang. Finbert: A pre-trained language model for financial communications. arXiv preprint arXiv:2006.08097, 2020.
- Shunyu Yao, Jeffrey Zhao, et al. React: Synergizing reasoning and acting in language models. In International Conference on Learning Representations, 2023.

A LIMITATIONS AND FUTURE DIRECTIONS

While VYNN AI demonstrates that agentic architectures can reliably integrate unstructured text into symbolic financial analysis, several limitations remain. These limitations arise directly from deliberate architectural choices described in Sections 3–5 and reflect broader research challenges for agent-based text-mining systems beyond finance.

A.1 CURRENT LIMITATIONS

Semantic Latency. As shown in Section 5, end-to-end latency is dominated by LLM-mediated semantic agents, particularly during large-scale news ingestion and event extraction. This follows from the architectural separation introduced in Section 4, where semantic interpretation is isolated from symbolic computation to preserve correctness and reproducibility. Cache-aware fan-out substantially reduces redundant computation for repeated queries, but reducing first-hit semantic latency without degrading extraction fidelity remains an open challenge, reflecting a broader tension between semantic depth and real-time responsiveness in LLM-based systems Dean & Barroso (2013); Kwon et al. (2023).

Limited Scalability Characterization. Our evaluation prioritizes real production workloads over synthetic stress testing. Consequently, we do not report throughput saturation points or tail-latency behavior under extreme concurrency. While cache-aware orchestration bounds amortized computation by construction, controlled scalability benchmarks remain necessary to characterize worst-case behavior, particularly given the non-linear variability of semantic workloads.

Non-Learning Supervisor Policy. To ensure determinism and safety, the Supervisor’s routing policy is rule-constrained and does not adapt from historical executions. This choice prioritizes reproducibility and auditability over optimization, but limits dynamic improvements in agent ordering or caching strategies. Enabling learning-based routing while preserving symbolic invariants represents a promising yet non-trivial research direction, intersecting with broader work on safe and constrained AI systems Amodei et al. (2016).

A.2 AGENTIC TEXT MINING BEYOND FINANCE

The primary contribution of VYNN AI lies not in the financial model itself, but in a transferable architectural pattern: semantic agents transform unstructured text into structured state, while symbolic modules enforce domain-specific correctness under explicit control. Key primitives include isolating semantic interpretation from deterministic reasoning, grounding extracted signals in explicit evidence spans, enforcing symbolic downstream computation, and bounding redundant work via cache-aware fan-out.

These principles generalize to domains such as healthcare (clinical notes and regulatory updates), legal analysis (contracts and case law), public policy (legislative and geopolitical developments), and supply-chain intelligence. In each case, agentic decomposition enables domain-specific semantic interpretation while mitigating risks associated with ungrounded language-model reasoning Bommasani et al. (2021).

A.3 INFORMATION LATENCY AND OVERSIGHT

Rather than eliminating information asymmetry in a strict economic sense, agent-based text-mining systems reduce the latency between public information release and structured interpretation Hong et al. (2000). In finance, this manifests as faster incorporation of news into valuation models; in other domains, it may enable earlier warnings in settings where delayed interpretation carries high cost.

Future work includes learning-enhanced Supervisors that optimize routing and caching decisions under symbolic constraints, as well as human-in-the-loop oversight mechanisms. Interactive review workflows, confidence-weighted explanations, and governance layers that leverage explicit state transparency represent promising directions at the intersection of agentic AI and reliable systems design Shinn et al. (2023).

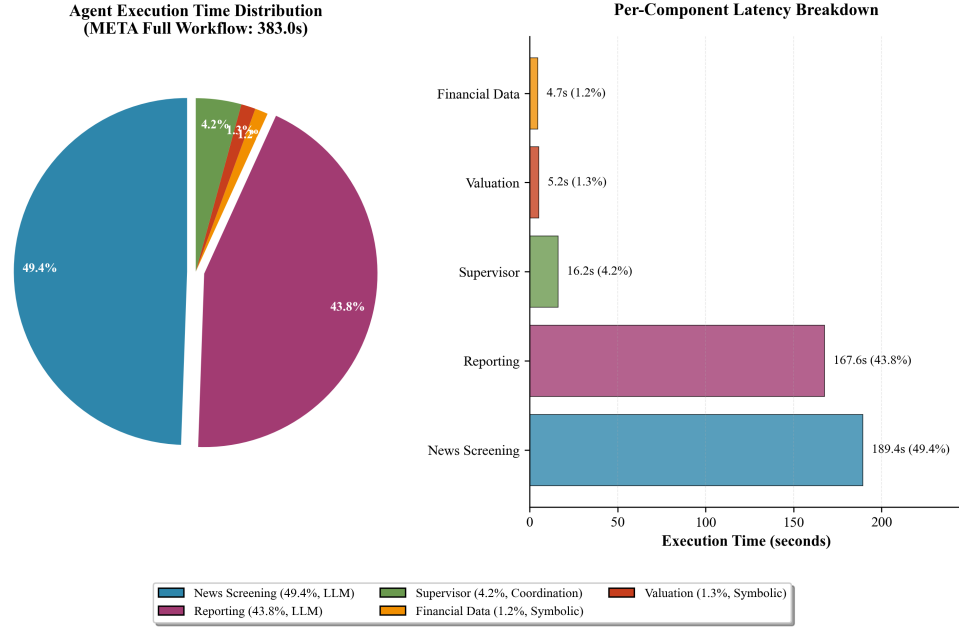


Figure 4: Per-agent execution time breakdown for a full META workflow. Semantic agents dominate total runtime, while symbolic agents incur minimal overhead.

A.4 CLOSING REMARKS

VYNN AI illustrates how agentic architectures can bridge unstructured text and structured decision-making. Although the present system targets financial analysis, its underlying principles—semantic decomposition, symbolic grounding, and cache-aware orchestration—are broadly applicable. We hope this work motivates further research on agentic systems that combine semantic flexibility with symbolic guarantees in domains where correctness, interpretability, and timeliness are essential.

B DETAILED RUNTIME ANALYSIS

This appendix provides additional runtime diagnostics that complement the aggregate latency results reported in Section 5. All measurements are derived from the same production executions described in Section 5, and are included here to provide deeper insight into agent-level behavior without overloading the main narrative.

B.1 PER-AGENT EXECUTION TIME BREAKDOWN

Figure 4 shows a per-agent execution time breakdown for a full META workflow. The results highlight a clear separation between semantic and symbolic components. The News Screening and Reporting agents, both LLM-mediated, account for the majority of end-to-end latency, while the Financial Data and Valuation agents incur minimal overhead. Supervisor coordination contributes a small but non-negligible fraction of total runtime.

This decomposition reinforces the architectural separation introduced in Section 4: semantic agents perform flexible but expensive text interpretation, while symbolic agents execute deterministic numerical computation with predictable cost. The explicit orchestration boundary ensures that increases in semantic workload do not propagate instability into valuation logic.

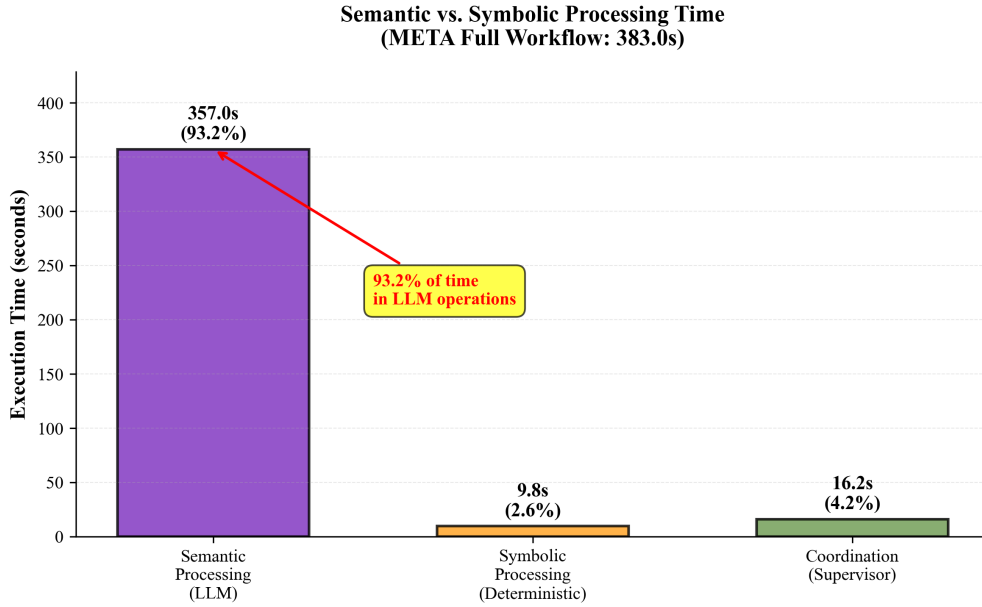


Figure 5: Semantic versus symbolic processing time for a full META workflow. LLM-mediated semantic processing dominates runtime, motivating cache-aware execution.

B.2 SEMANTIC VERSUS SYMBOLIC PROCESSING TIME

Figure 5 aggregates execution time by processing type. Over **93%** of total runtime is attributable to LLM-mediated semantic processing, while symbolic valuation and supervisory coordination together account for a small fraction of execution time.

This result motivates the cache-aware fan-out strategies described in Section 5. Because symbolic computation is both fast and deterministic, repeated executions primarily benefit from caching semantic artifacts such as extracted events and synthesized narratives. As a result, effective latency reductions scale with semantic reuse rather than numerical shortcutting.

B.3 EVALUATION EXPERIMENT SUMMARY

Exp.	Objective	#Runs	Tickers	Metrics
1	End-to-end latency	5	META, AAPL, GOOGL, AMZN	Latency (total / agent)
2	Cache amortization	1 (derived)	META	Effective latency
3	Reproducibility & stability	9	NVDA, AAPL, MSFT	Exact match, CV
4	Qualitative correctness	3	META, NVDA, AAPL	Event grounding

Table 1: Summary of evaluation experiments conducted on the production VYNN AI deployment. All results are obtained from real system executions; no synthetic workloads are used.

C DESIGN RATIONALE AND EXECUTION GUARANTEES

This appendix elaborates on architectural design choices that prioritize determinism, interpretability, and execution stability in VYNN AI. While Section 4 describes the mechanics of the multi-agent pipeline, the discussion here clarifies why alternative approaches were rejected and how system-level guarantees are enforced in practice.

C.1 REJECTION OF MONOLITHIC PROMPTING

End-to-end prompting strategies that embed multi-step reasoning, numerical computation, and control flow within a single language-model invocation were explicitly avoided. In practice, such approaches exhibit unreliable long-horizon reasoning, prompt sensitivity, and limited intermediate verifiability. These limitations are particularly problematic in financial analysis, where numerical correctness, auditability, and reproducibility are essential.

By decomposing execution into explicit agent stages with structured state transitions, VYNN AI ensures that intermediate artifacts can be inspected, validated, and reused. This design aligns with established systems principles favoring modular execution and observability over opaque monolithic inference.

C.2 AVOIDING LLM-ONLY VALUATION

Delegating valuation logic to language models introduces numerical instability and undermines reproducibility. Even minor arithmetic inconsistencies can propagate into materially incorrect valuation outputs. VYNN AI therefore restricts LLM usage to semantic tasks—such as event extraction and narrative synthesis—while implementing all numerical computation symbolically.

This separation guarantees exact reproducibility of valuation outputs under identical inputs and enables precise auditing of intermediate results. It also allows valuation logic to evolve independently of semantic modeling, reducing coupling between language-model behavior and financial correctness.

C.3 LIMITATIONS OF BLACK-BOX TOOL CALLING

While tool-calling frameworks extend language models with external capabilities, they often obscure control flow, state transitions, and termination conditions. Such opacity complicates debugging and makes it difficult to enforce execution invariants at scale.

In contrast, VYNN AI adopts an explicit graph-based orchestration model in which all admissible execution paths are defined a priori. The Supervisor enforces structural dependencies, bounded agent invocation, and deterministic termination at the orchestration level, with per-agent timeouts enforced in production. This design enables transparent debugging, controlled extensibility, and predictable execution behavior.

C.4 SYSTEM-LEVEL GUARANTEES

Taken together, these design choices ensure the following system-level properties:

- **Deterministic termination** at the orchestration level via acyclic computation graphs with bounded agent invocations;
- **Symbolic reproducibility** of all numerical outputs under identical inputs;
- **Interpretability** through explicit state tracking and evidence-grounded semantic artifacts;
- **Stability** under paraphrased inputs due to structured routing and bounded parameter updates;
- **Scalability** through cache-aware fan-out and isolated worker execution.

These guarantees position VYNN AI as a reliable agentic framework for high-stakes, text-driven decision-making domains where correctness, transparency, and operational robustness are essential.